

Understand Stack Reference vs Heap Object (Beginner Level + Interview POV)

Interview Goal: This is one of the most important Java memory concepts. Interviewers usually ask it before discussing **String, Arrays, Objects, Garbage Collection, and Pass-by-Value**.

Learning Objectives

After this topic, you will be able to:

- Understand what the **Stack** and **Heap** are.
 - Know where objects and reference variables are stored.
 - Understand how references point to objects.
 - Answer common interview questions.
-

1. Stack vs Heap

Java memory is simplified into two important areas:

Stack	Heap
Stores local variables and references	Stores objects and arrays
Faster access	Larger memory area
Automatically cleaned when a method ends	Objects remain until no longer referenced

Beginner Tip: Don't think of the stack as storing the entire object—it stores **only the reference (address)** to the object.

2. Primitive Variable vs Object

Primitive Variable

```
int age = 22;
```

Memory:

```
Stack  
-----  
age = 22
```

The actual value is stored directly.

Object

```
Student s = new Student();
```

Memory:

```
Stack                Heap  
-----            -----  
s -----> Student Object
```

- **Stack:** Stores the reference variable s.
 - **Heap:** Stores the actual Student object.
-

3. More Than One Reference

```
Student s1 = new Student();  
Student s2 = s1;
```

Memory:

Stack		Heap
-----		-----
s1 -----\		
	----->	Student Object
s2 -----/		

Both variables refer to the **same object**.

Interview Insight

If one reference changes the object's state:

```
s1.name = "Ali";
```

Then:

```
System.out.println(s2.name);
```

Output:

```
Ali
```

Why?

Because both references point to the **same object**, not because Java is pass-by-reference.

4. Creating a New Object

```
Student s1 = new Student();
Student s2 = new Student();
```

Memory:

Stack	Heap
-----	-----
s1 ----->	Student Object A
s2 ----->	Student Object B

Now both references point to **different objects**.

5. Null Reference

```
Student s = null;
```

Memory:

```
Stack  
-----  
s = null
```

There is **no object** in the Heap.

Trying to use it:

```
s.name = "Ali";
```

Results in:

```
NullPointerException
```

Interviewer's Reasoning

Interviewers don't ask:

"Where is the object stored?"

They're checking if you understand **how references work**.

For example:

```
Student s1 = new Student();  
Student s2 = s1;
```

Question:

How many objects are created?

Answer:

Only **one**.

Many beginners incorrectly answer **two** because there are two variables.

Common Misconceptions

✗ Every reference creates a new object.

```
Student s2 = s1;
```

No new object is created.

✗ The Stack stores the whole object.

Incorrect.

The **Stack stores only the reference**, while the **Heap stores the object**.

✗ null means an empty object.

Incorrect.

null means **no object is being referenced**.

Interview Comparison

Primitive vs Reference Variable

Primitive	Reference
Stores actual value	Stores address/reference
int, double, char	Objects, Arrays, String
Stored on Stack (local variables)	Reference on Stack, object on Heap

One Object vs Two Objects

```
Student s1 = new Student();  
Student s2 = s1;
```

✓ Objects Created:

1

```
Student s1 = new Student();  
Student s2 = new Student();
```

✓ Objects Created:

2

Tricky Interview Questions

Q1. Where is the object stored?

Answer:

In the **Heap**.

Q2. Where is the reference variable stored?

Answer:

In the **Stack** (for local variables).

Q3. Does this create two objects?

```
Student s1 = new Student();  
Student s2 = s1;
```

Answer:

No, only **one object**.

Q4. What happens when all references to an object are removed?

```
Student s = new Student();  
s = null;
```

Answer:

The object becomes **eligible for Garbage Collection** because nothing references it anymore.

Output Question

```
class Student {
    String name;
}

public class Main {

    public static void main(String[] args) {

        Student s1 = new Student();
        Student s2 = s1;

        s1.name = "Ali";

        System.out.println(s2.name);
    }
}
```

Output

Ali

Reason: Both s1 and s2 point to the same object.

Cheat Sheet

Concept	Key Point
Stack	Stores local variables and references
Heap	Stores objects and arrays
Primitive	Value stored directly
Reference	Stores object's address/reference
new	Creates an object in the Heap
null	No object is referenced

Top Interview Questions

1. What is the difference between Stack and Heap?
 2. Where are objects stored?
 3. Where are reference variables stored?
 4. How many objects are created in `Student s2 = s1;`?
 5. What does null mean?
 6. Why do changes through one reference appear through another?
 7. When does an object become eligible for Garbage Collection?
-

Interview Insight (Very Common)

A common interview question is:

"If the object is in the Heap, why can I access it using [`s.name`](#)?"

A strong answer is:

"The variable `s` doesn't contain the object itself; it contains a **reference** to the object in the Heap. Java automatically follows that reference to access the object's fields and methods."

This shows that you understand the relationship between **references** and **objects**, which is essential for topics like object assignment, method calls, arrays, String, and Java's pass-by-value behavior.